

PROJECT 5 REPORT

Junrui Ding, Junyao Han

Overall Project Description

This project explores building, training, analyzing, and modifying deep neural networks for image recognition. Using the MNIST handwritten digit dataset, we built a convolutional neural network (CNN) from scratch, examined its learned filters, applied transfer learning to recognize Greek letters, re-implemented the network using transformer layers, and conducted an ablation study to evaluate how different architectural choices affect performance.

Task 1: Build and Train a Network to Recognize Digits

For this task, we built a CNN using PyTorch and trained it on the MNIST digit dataset (60,000 training images, 10,000 test images, each 28×28 grayscale).

The network architecture consists of:

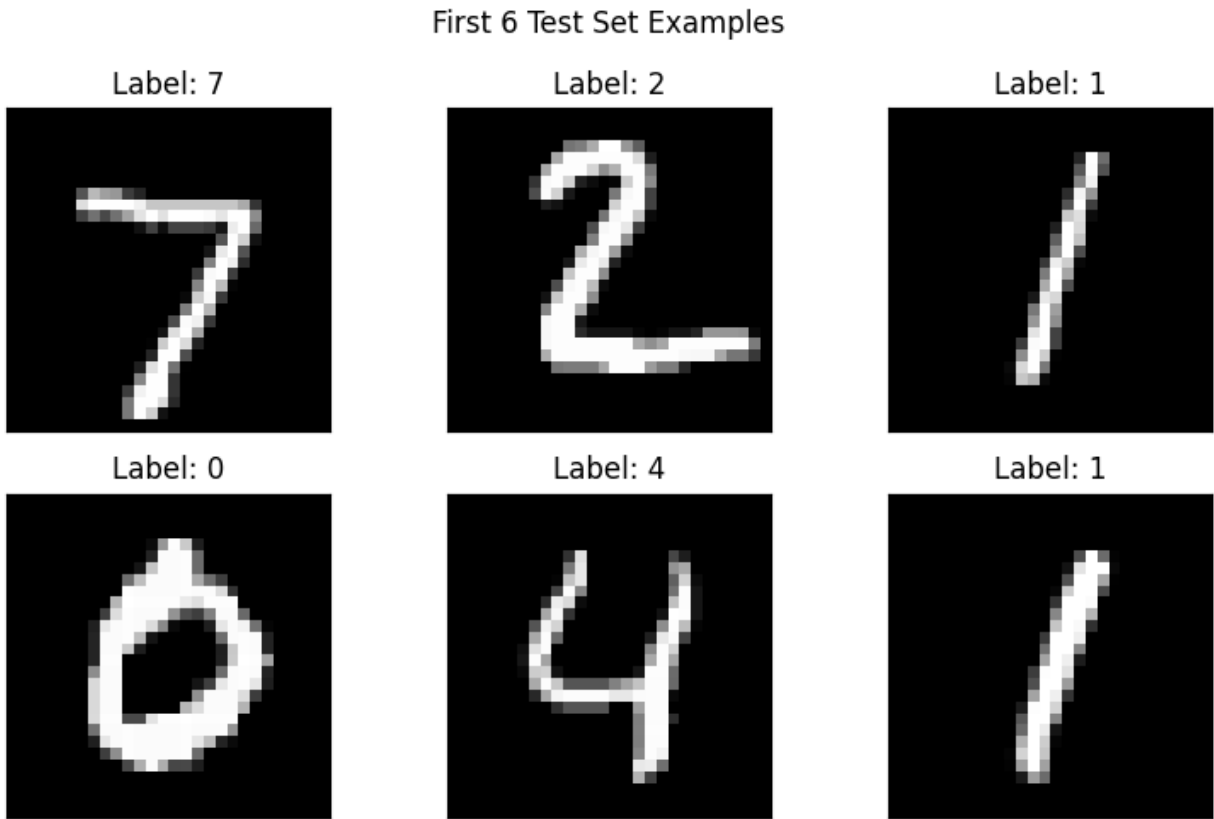
- A convolutional layer with 10 5×5 filters
- A max pooling layer (2×2) with ReLU activation
- A second convolutional layer with 20 5×5 filters
- A dropout layer (rate=0.5)
- A second max pooling layer with ReLU
- A fully connected layer with 50 nodes and ReLU
- A final fully connected layer with 10 nodes and log-softmax output

The model was trained using SGD with momentum=0.5, learning rate=0.01, and batch size=64 for 5 epochs.

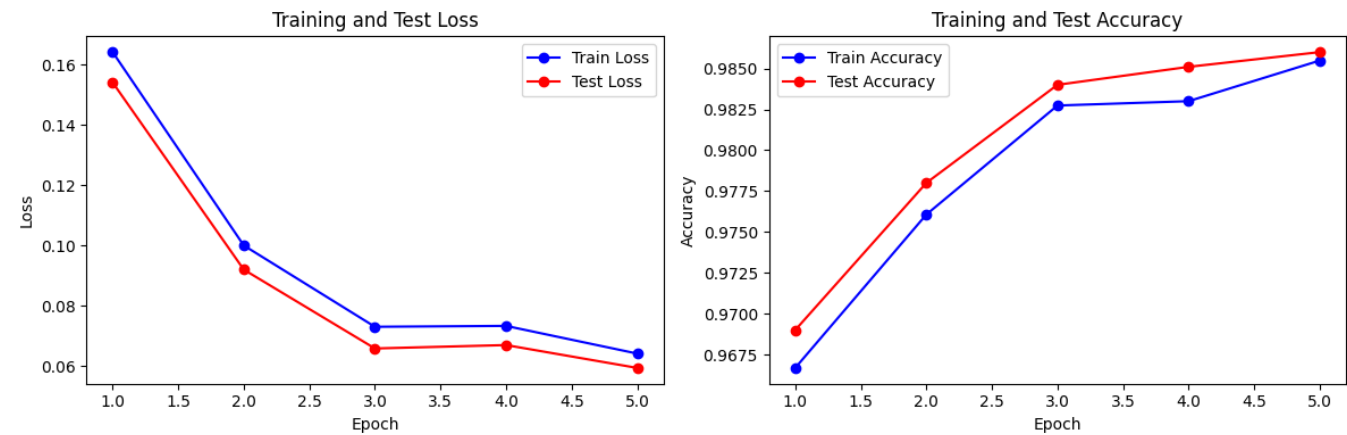
The network architecture is illustrated below:

```
Input (1×28×28 grayscale image)
→ Conv2d(1→10, 5×5) → MaxPool2d(2×2) → ReLU
→ Conv2d(10→20, 5×5) → Dropout(0.5) → MaxPool2d(2×2) → ReLU
→ Flatten → Linear(320→50) → ReLU
→ Linear(50→10) → LogSoftmax
→ Output (10 digit classes)
```

The first 6 examples from the test set are shown below:



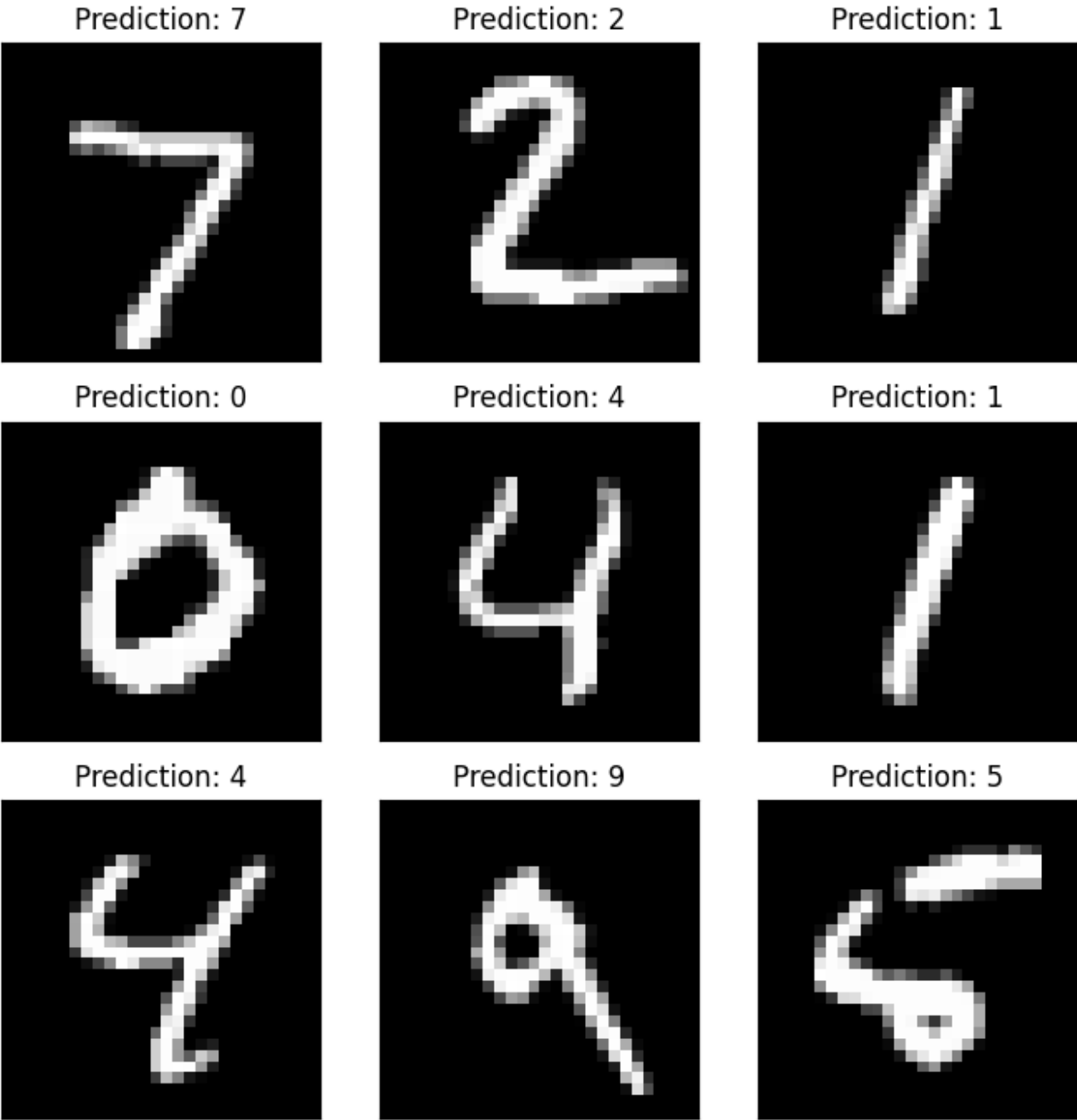
As shown in the figure below, both training and test loss decreased steadily across epochs, while accuracy improved from ~96% to ~98.6% on the test set. The test accuracy consistently matched or slightly exceeded training accuracy, indicating the model generalized well without overfitting.



We then ran the trained model on the first 10 examples of the test set. The model correctly classified all 10 examples. The output log-probabilities, predicted labels, and correct labels are shown in the table below:

```
(venv) jrd@JunruideMacBook-Pro src % python mnist_test.py
Index  Output Values                                     Predicted  Correct
-----
0      ['-10.18', '-10.94', '-6.68', '-8.12', '-13.43', '-12.91', '-18.77', '-0.00', '-10.43', '-9.47'] 7      7
1      ['-6.67', '-5.40', '-0.01', '-9.30', '-13.14', '-14.59', '-9.48', '-12.03', '-7.66', '-15.85'] 2      2
2      ['-6.35', '-0.03', '-5.75', '-8.35', '-4.56', '-7.32', '-6.24', '-4.83', '-6.85', '-8.85'] 1      1
3      ['-0.00', '-12.48', '-7.34', '-9.71', '-13.07', '-10.16', '-7.82', '-8.49', '-10.20', '-7.93'] 0      0
4      ['-9.35', '-11.61', '-9.77', '-14.03', '-0.00', '-11.32', '-10.56', '-8.61', '-11.81', '-6.08'] 4      4
5      ['-7.61', '-0.01', '-7.50', '-10.26', '-5.33', '-9.19', '-8.24', '-5.85', '-8.33', '-10.24'] 1      1
6      ['-13.76', '-7.10', '-10.18', '-10.94', '-0.02', '-7.05', '-11.08', '-5.73', '-5.14', '-5.29'] 4      4
7      ['-9.55', '-7.14', '-4.72', '-4.33', '-2.97', '-5.17', '-11.28', '-4.74', '-4.52', '-0.11'] 9      9
8      ['-9.32', '-10.34', '-7.48', '-9.53', '-7.64', '-0.03', '-3.60', '-9.17', '-5.63', '-6.33'] 5      5
9      ['-9.34', '-12.69', '-9.73', '-6.90', '-5.73', '-9.54', '-13.71', '-2.58', '-5.01', '-0.09'] 9      9
0.png: Predicted = 0
1.png: Predicted = 6
2.png: Predicted = 2
3.png: Predicted = 2
4.png: Predicted = 8
5.png: Predicted = 5
6.png: Predicted = 1
7.png: Predicted = 7
8.png: Predicted = 8
9.png: Predicted = 1
```

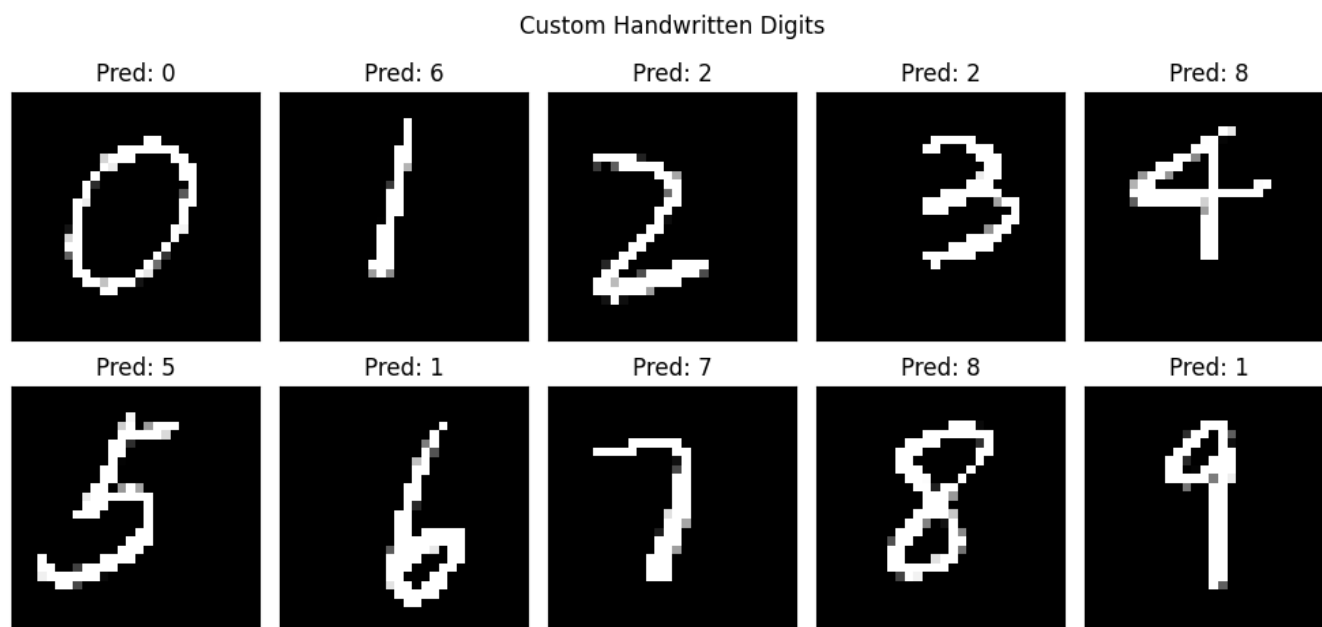
The first 9 test set predictions are shown below in a 3x3 grid:



Task 1F: Testing on Custom Handwritten Digits

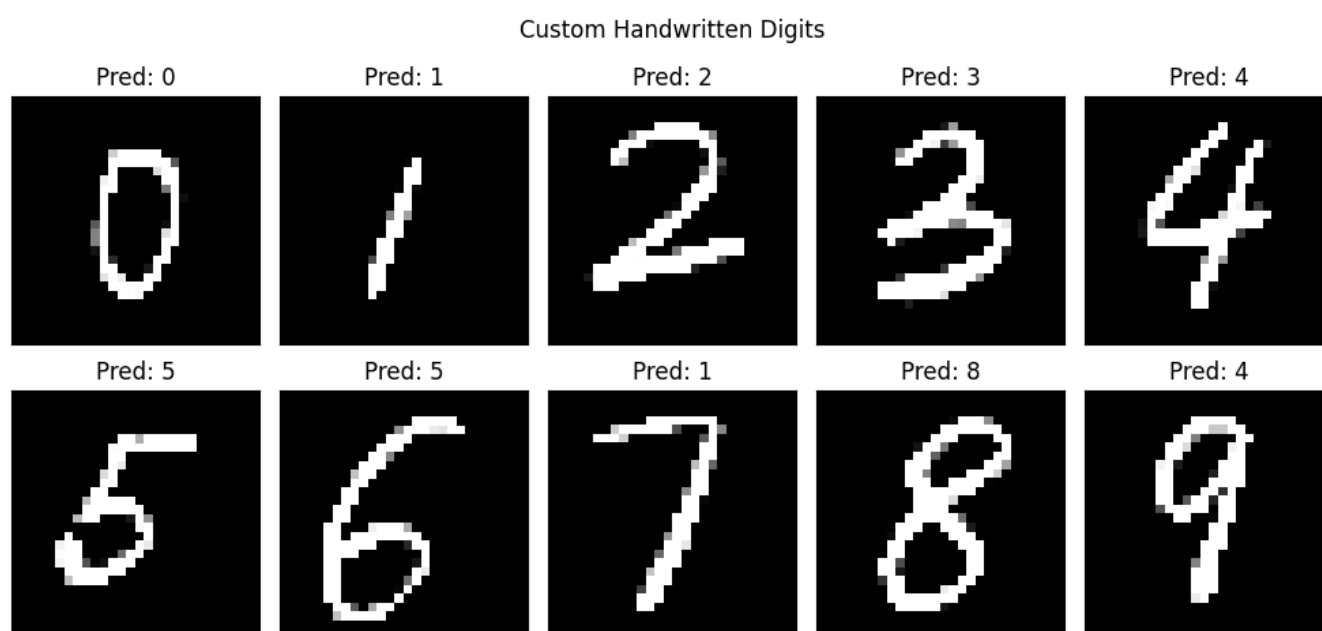
We tested the network on handwritten digits [0–9] written on a tablet using a stylus in Notability. Images were individually cropped, and the preprocessing pipeline automatically inverted intensities to match MNIST format (white digit on black background).

Version 1 (original handwriting style) — 4/10 accuracy:



Errors: 1→6, 4→8, 6→1, 9→1.

Version 2 (revised to closer match MNIST style) — 6/10 accuracy:



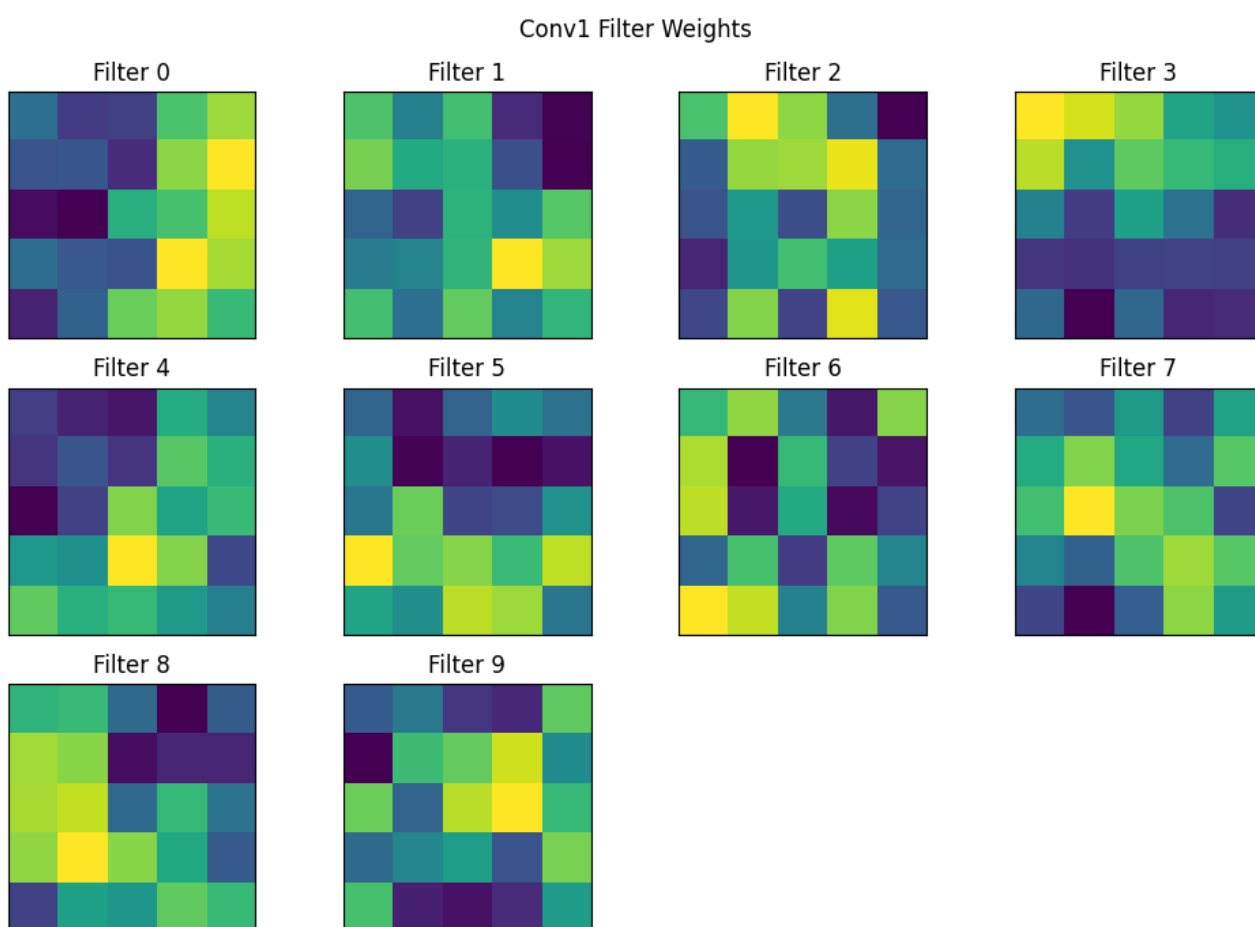
Errors: 6→5, 7→1, 9→4.

Analysis of the preprocessed images confirmed correct formatting (white digit on black background, centered), so errors were attributed to handwriting style differences from the MNIST training distribution. In

v1, a heavily slanted 1, closed-top 4, and stylized 6 and 9 caused misclassifications. In v2, revising to straighter strokes and an open-top 4 improved accuracy from 4/10 to 6/10. The remaining misclassified 7 (predicted as 1) likely resulted from a thin horizontal stroke at the top — without a prominent horizontal feature, the dominant visual pattern becomes a single diagonal line, which closely resembles a 1 in the MNIST distribution. The errors suggest the model is sensitive to stroke weight and style variations not well-represented in the training data.

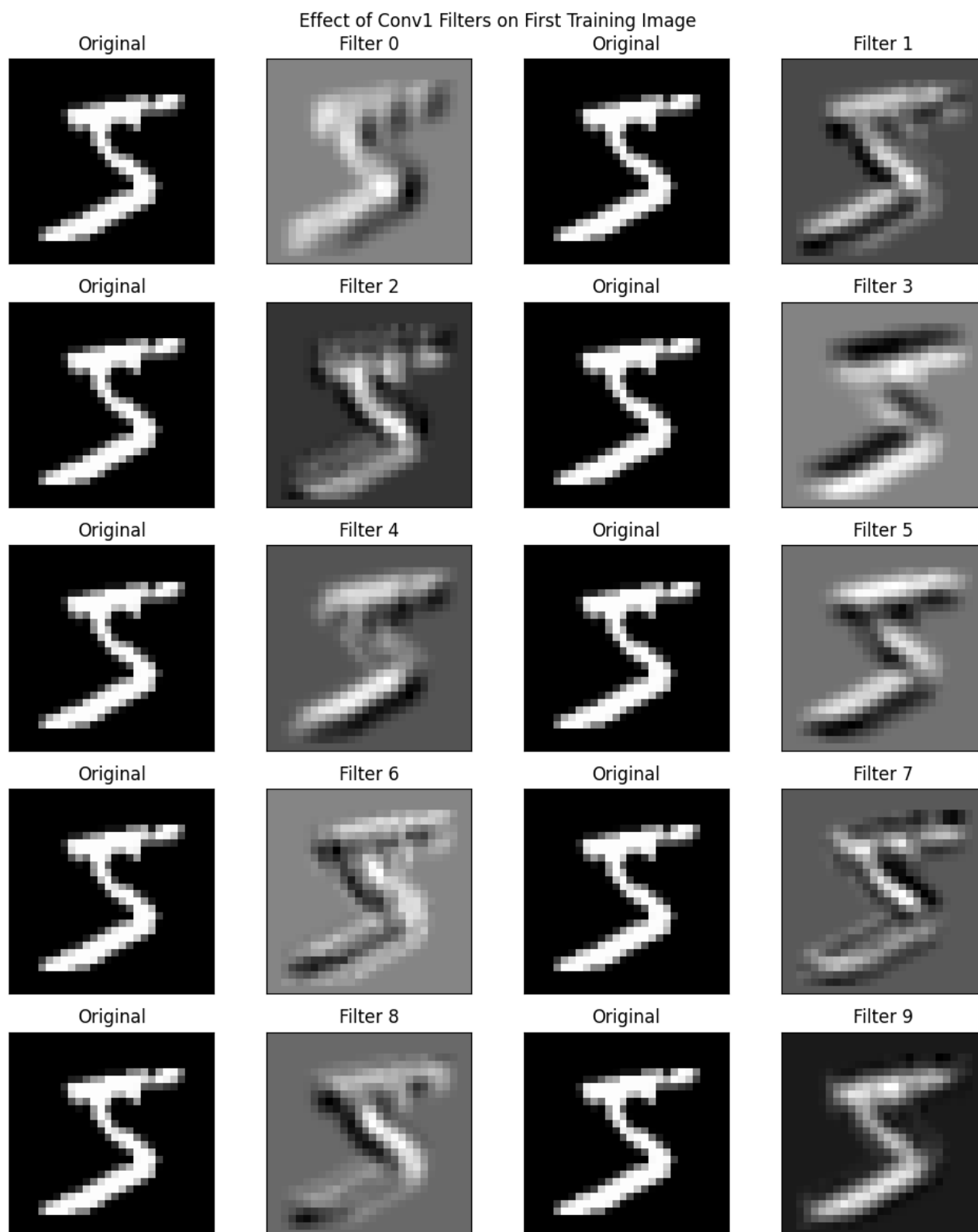
Task 2: Examine the Network

We loaded the trained model and printed its structure to identify layer names. We then extracted the weights of the first convolutional layer (`conv1`), which has shape `[10, 1, 5, 5]` — 10 filters, each 5×5.



The filters show varied spatial patterns with no two filters identical, indicating the network has learned to detect different local features. Some filters (e.g., Filter 0, 7) have strong localized bright spots suggesting they respond to specific corners or blob-like structures. Others (e.g., Filter 3, 6) show more gradient-like patterns across the spatial extent, likely responding to oriented edges or strokes. This diversity is expected and desirable — each filter specializes in a different aspect of the input.

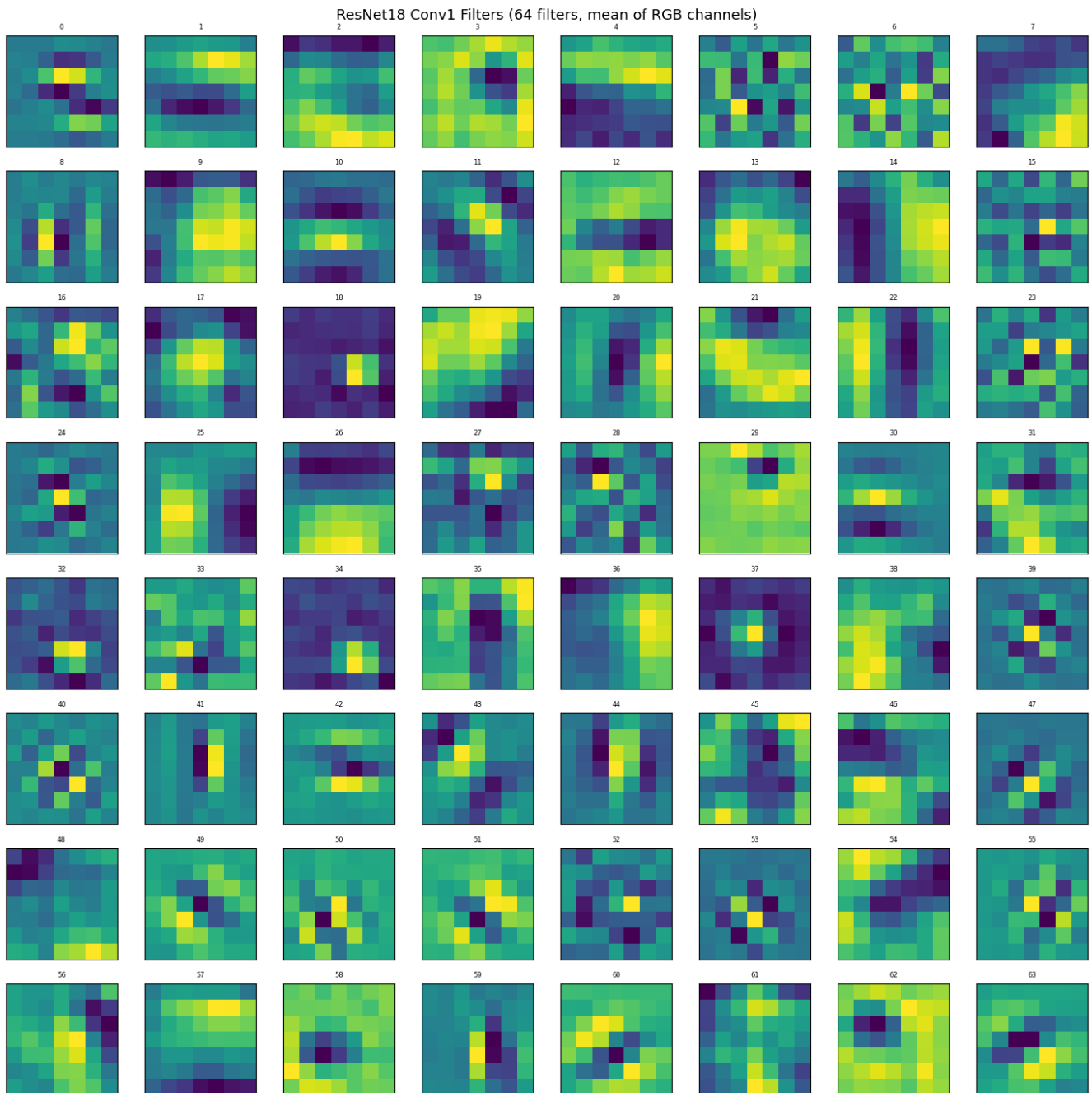
We then applied all 10 filters to the first training image (digit "5") using OpenCV's `filter2D` function:



The results make sense given the filter shapes. Some filters produce smoothing/brightening effects (e.g., Filter 0), preserving the overall stroke structure. Others produce strong edge enhancement (e.g., Filter 2, 6), highlighting the boundaries and directional strokes of the digit. Filters with opposing positive/negative weights (e.g., Filter 3) produce a gray background with sharp contrast at stroke edges, consistent with edge detection behavior.

Extension: Examining a Pre-trained ResNet18

As an extension of Task 2, we loaded a pretrained ResNet18 model from PyTorch and examined the weights of its first convolutional layer. ResNet18 has 64 filters in `conv1`, each of size 7×7 with 3 input channels. To visualize them in a simple way, we averaged each filter across the RGB channels and plotted the 64 resulting filters in an 8×8 grid.

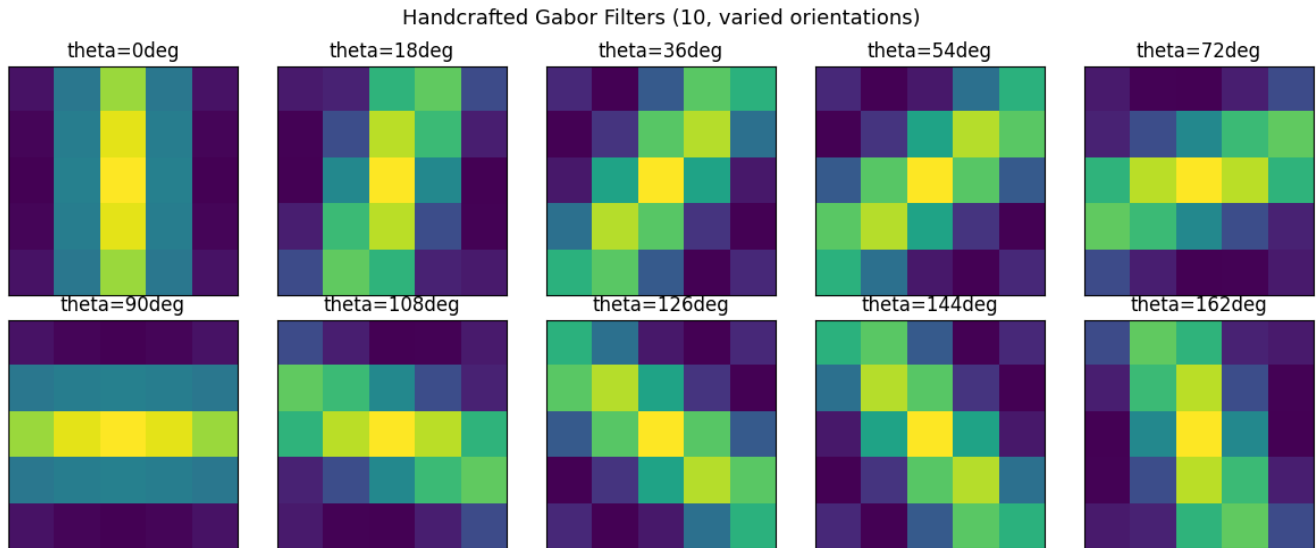


The filters are noticeably more complex than those learned by our MNIST CNN. While the MNIST filters mainly capture simple stroke fragments, blobs, and edge responses in grayscale, the ResNet18 filters show stronger directional structure and more diverse spatial patterns. Several filters resemble Gabor-like edge detectors, with oriented bright and dark regions that would respond to edges or texture transitions in natural images.

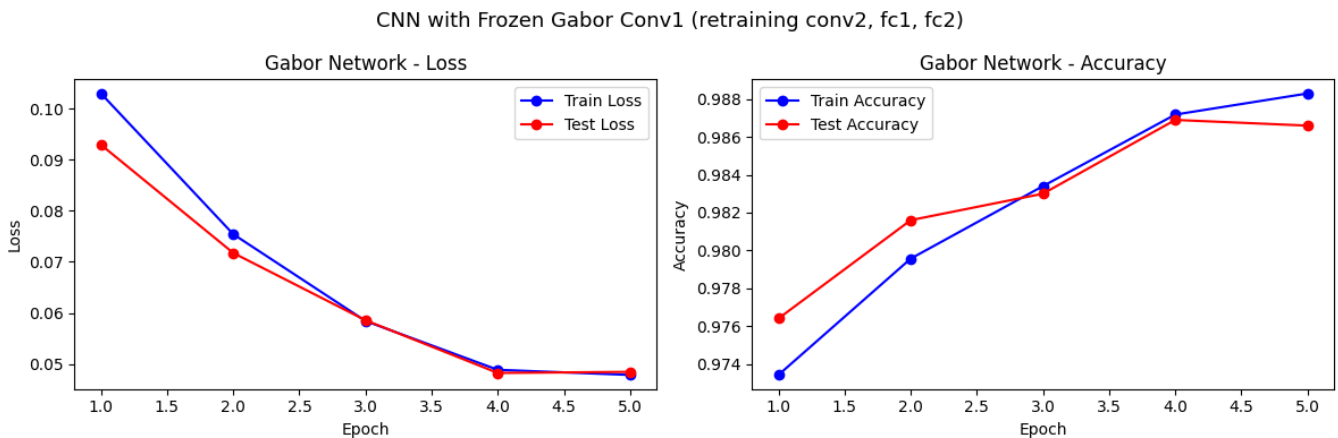
This difference is expected because ResNet18 was pretrained on ImageNet, which contains a much broader variety of objects and visual patterns than MNIST. The pretrained filters therefore capture more general-purpose low-level image features, while the MNIST filters are more specialized for handwritten digit recognition.

Extension: Replacing the First Layer with Gabor Filters

To explore whether hand-designed filters can substitute for learned filters, we replaced the first convolutional layer of the MNIST CNN with a bank of 10 fixed 5×5 Gabor filters. The filters were generated with different orientations and then assigned directly to `conv1.weight`. This layer was frozen, and only the remaining layers (`conv2`, `fc1`, and `fc2`) were retrained for 5 epochs.



The modified network trained successfully and reached strong performance:



The final test accuracy of the Gabor model was **98.66%**, compared with **98.79%** for the original fully trained MNIST model. The Gabor-based model was therefore only **0.13 percentage points lower** than the learned-filter model.

This is a strong result. Gabor filters are classical edge detectors and are often considered biologically inspired approximations of early visual processing. The fact that the frozen Gabor first layer performed almost as well as the learned first layer suggests that the first CNN layer on MNIST is primarily learning edge- and stroke-oriented features anyway. At the same time, the slightly better performance of the original model indicates that learned filters still retain some advantage by adapting directly to the statistics of the training data.

Task 3: Transfer Learning on Greek Letters

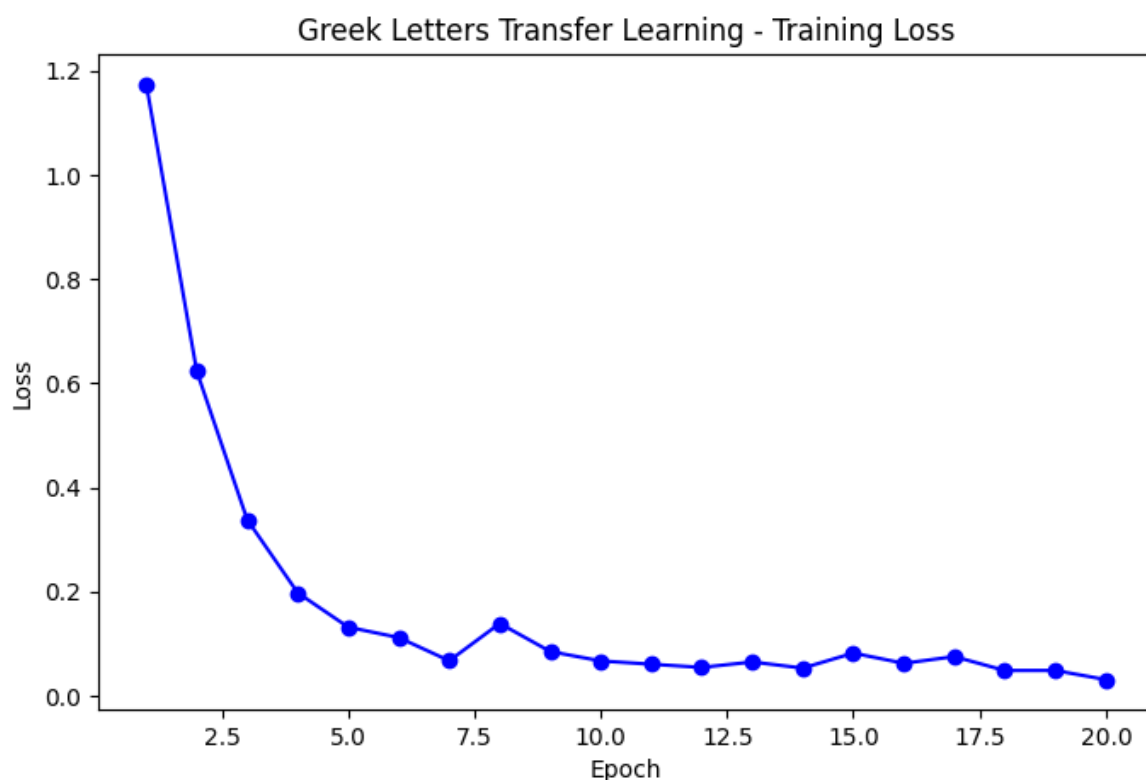
The Greek letter dataset was provided by the course instructor and consists of 27 labeled images (9 per class: alpha, beta, gamma), each 133×133 pixels in color. Images were preprocessed using a custom `GreekTransform` pipeline: converted to grayscale, scaled and center-cropped to 28×28, and intensity-inverted to match the MNIST format (white strokes on black background). The dataset was loaded using PyTorch's `ImageFolder` class.

For this task, we adapted the pretrained MNIST CNN to classify three Greek letters: alpha, beta, and gamma, using only 27 training examples (9 per class). We froze all network weights except the final fully connected layer, which was replaced with a new 3-node output layer. Only this new layer (153 parameters) was trained — the rest of the network retained its MNIST-learned features.

The modified network printout is shown below:

```
MyNetwork(  
  (conv1): Conv2d(1, 10, kernel_size=(5, 5), stride=(1, 1))  
  (conv2): Conv2d(10, 20, kernel_size=(5, 5), stride=(1, 1))  
  (dropout): Dropout(p=0.5, inplace=False)  
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
  ceil_mode=False)  
  (fc1): Linear(in_features=320, out_features=50, bias=True)  
  (fc2): Linear(in_features=50, out_features=3, bias=True)  
)
```

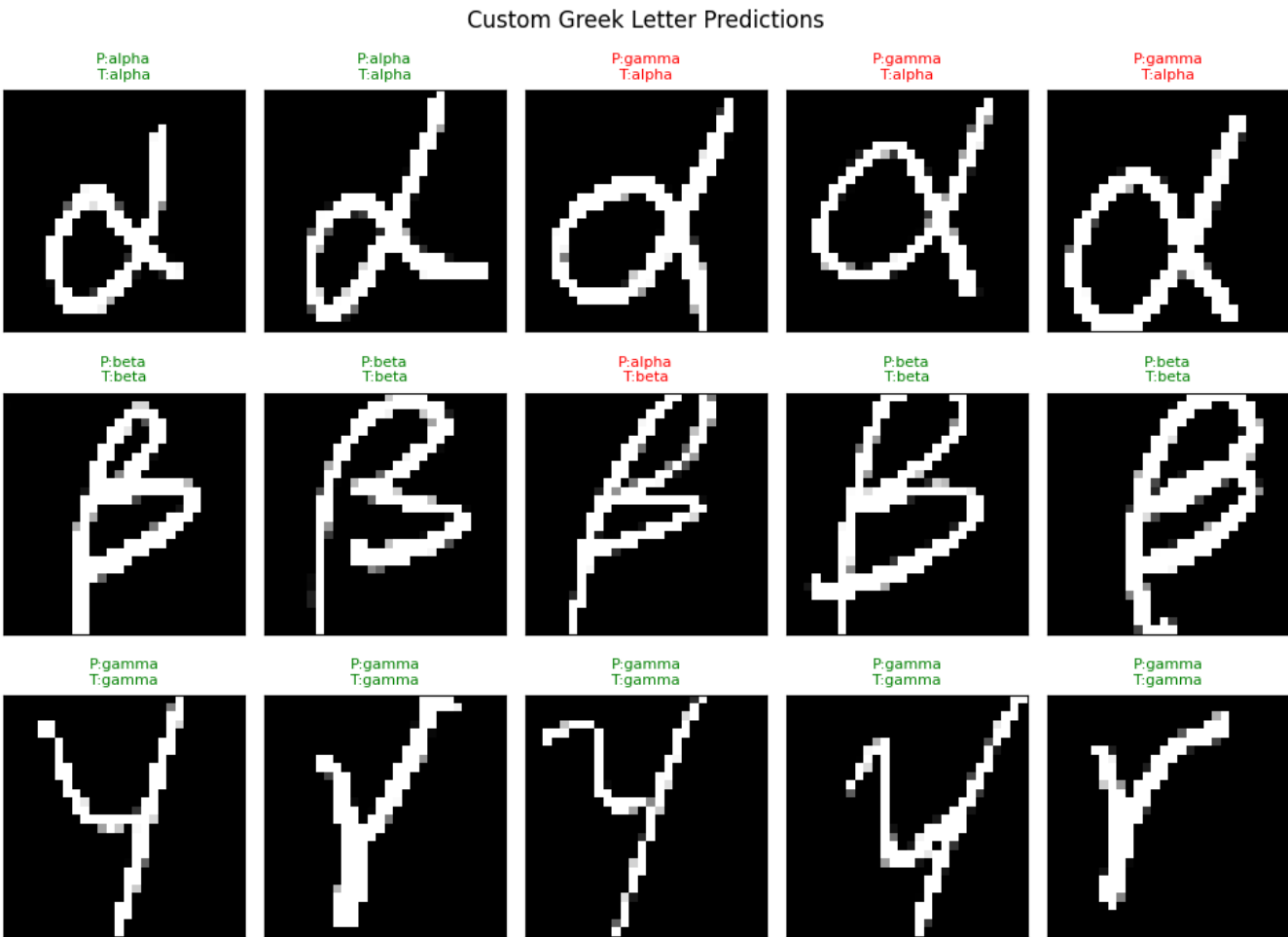
The only change from the original network is `fc2`: output nodes reduced from 10 to 3 (alpha, beta, gamma). All other layers are frozen.



As shown in the figure, the loss dropped sharply in the first 5 epochs and continued to decrease gradually, approaching near-zero by epoch 20. The network reached near-perfect classification on the training set within approximately 5 epochs. This demonstrates the power of transfer learning — the convolutional features learned on MNIST digits (edges, strokes, curves) transferred effectively to the Greek letter recognition task, even with only 27 examples.

The network reached 100% accuracy (27/27) on the training set by epoch 9 and maintained it through epoch 20. The rapid convergence with only 27 examples confirms that the frozen convolutional features were already well-suited for this task.

We also tested the model on our own handwritten alpha, beta, and gamma symbols (5 images per class, 15 total). The model achieved **13/15 accuracy (87%)**. The two misclassified examples were alpha images predicted as gamma, likely due to writing style variations in the loop shape of alpha. All beta and gamma examples were correctly classified.

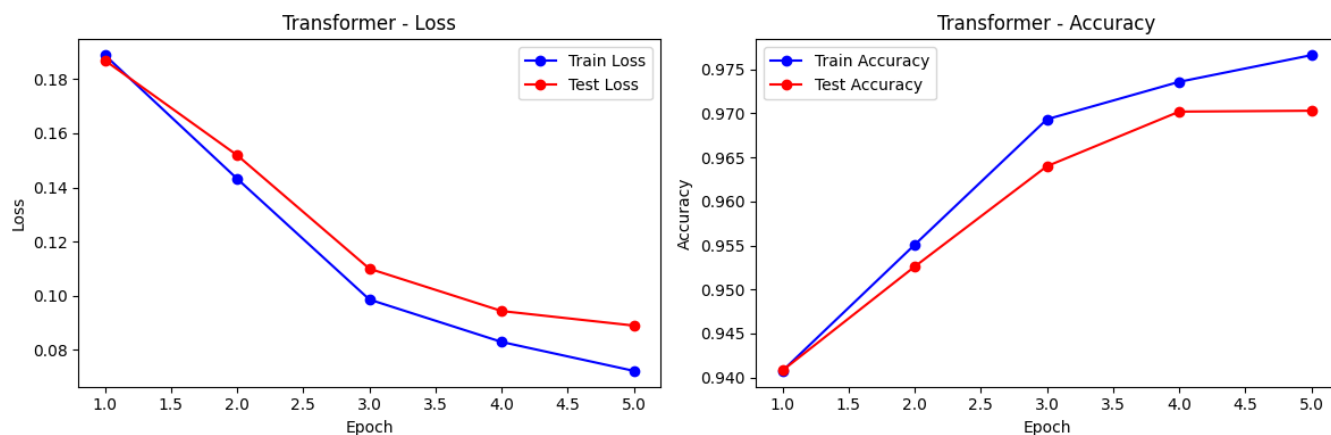


Green titles indicate correct predictions; red indicates misclassification.

Task 4: Re-implement with Transformer Layers

For this task, we replaced the CNN with a Vision Transformer (ViT)-style model. The 28×28 image is divided into 16 non-overlapping 7×7 patches. Each patch is linearly embedded into a 64-dimensional token, positional embeddings are added, and the sequence is processed by 2 transformer encoder layers (4

attention heads, MLP dimension 128). The output tokens are averaged to produce a single representation, which is passed through a classification head (Linear $\rightarrow$ ReLU $\rightarrow$ Linear with 10 outputs).

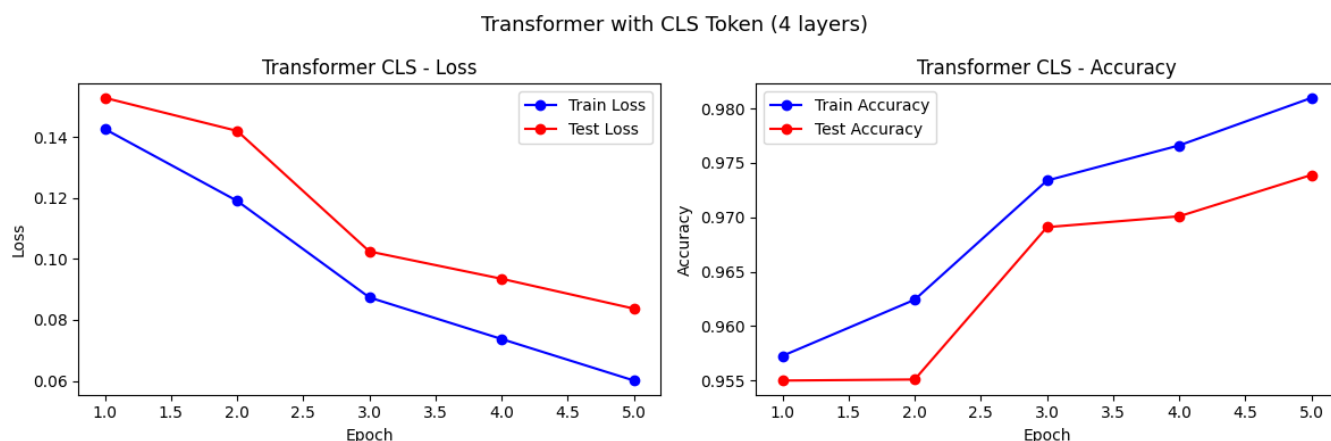


The transformer model achieved **97.1% test accuracy** after 5 epochs, compared to **98.6%** for the CNN. The transformer trained noticeably slower on CPU due to the attention computation overhead. The gap in accuracy is expected — transformers generally require more data and longer training to match CNNs on small datasets like MNIST, where local spatial structure (well-captured by convolutions) is the dominant feature.

Extension: Transformer with CLS Token

For this extension, we modified the transformer model from Task 4 by adding a learnable CLS token and increasing the encoder depth from 2 layers to 4 layers. Instead of averaging all patch tokens at the end of the transformer, the CLS token was prepended to the sequence and used as the single representation for classification.

The modified model contained **147,850 parameters** and was trained for 5 epochs on MNIST.



The model achieved **97.39% test accuracy** after 5 epochs, slightly higher than the **97.1%** test accuracy of the original mean-pooling transformer. The improvement was modest, but it suggests that the CLS token provided a slightly stronger global representation for classification.

This result makes sense conceptually. Mean pooling treats all patch tokens equally, while the CLS token is trained to gather the most useful information across the full image through self-attention. On a simple dataset like MNIST, both methods work well and the difference remains small. Still, the CLS token version is closer to the standard ViT design and may offer larger benefits on more complex datasets.

Task 5: Design Your Own Experiment

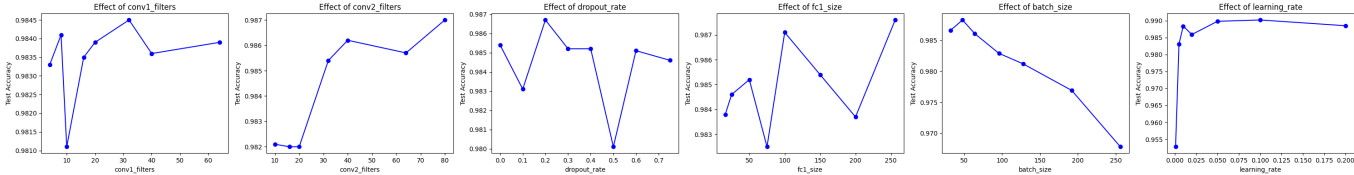
We conducted an ablation study on the CNN architecture using a linear search strategy — varying one dimension at a time while keeping others fixed at the best value found so far. Each configuration was trained for 3 epochs and evaluated on the test set. A total of 60 network variants were evaluated across 6 dimensions.

Dimensions explored:

- 1. conv1_filters: [4, 8, 10, 16, 20, 32, 40, 64]
- 2. conv2_filters: [10, 16, 20, 32, 40, 64, 80]
- 3. dropout_rate: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.75]
- 4. fc1_size: [16, 25, 50, 75, 100, 150, 200, 256]
- 5. batch_size: [32, 48, 64, 96, 128, 192, 256]
- 6. learning_rate: [0.001, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2]

Hypotheses (before running):

- conv1/conv2 filters: more filters → higher accuracy up to a point, then diminishing returns
- dropout_rate: moderate dropout helps generalization; very high dropout hurts
- fc1_size: optimal middle ground — too small underfits, too large may overfit in 3 epochs
- batch_size: smaller batches → noisier gradients but often better generalization
- learning_rate: too low = slow convergence in 3 epochs; too high = unstable training



Results:

Dimension	Value	Test Accuracy
conv1_filters	4	98.33%
conv1_filters	8	98.41%
conv1_filters	10	98.11%
conv1_filters	16	98.35%
conv1_filters	20	98.39%
conv1_filters	32	98.45%
conv1_filters	40	98.36%
conv1_filters	64	98.39%
conv2_filters	10	98.21%
conv2_filters	16	98.20%

Dimension	Value	Test Accuracy
conv2_filters	20	98.20%
conv2_filters	32	98.54%
conv2_filters	40	98.62%
conv2_filters	64	98.57%
conv2_filters	80	98.70%
dropout_rate	0.0	98.54%
dropout_rate	0.1	98.31%
dropout_rate	0.2	98.67%
dropout_rate	0.3	98.52%
dropout_rate	0.4	98.52%
dropout_rate	0.5	98.01%
dropout_rate	0.6	98.51%
dropout_rate	0.75	98.46%
fc1_size	16	98.38%
fc1_size	25	98.46%
fc1_size	50	98.52%
fc1_size	75	98.25%
fc1_size	100	98.71%
fc1_size	150	98.54%
fc1_size	200	98.37%
fc1_size	256	98.76%
batch_size	32	98.66%
batch_size	48	98.83%
batch_size	64	98.61%
batch_size	96	98.29%
batch_size	128	98.12%
batch_size	192	97.69%
batch_size	256	96.78%
learning_rate	0.001	95.28%
learning_rate	0.005	98.31%

Dimension	Value	Test Accuracy
learning_rate	0.01	98.84%
learning_rate	0.02	98.59%
learning_rate	0.05	98.98%
learning_rate	0.1	99.02%
learning_rate	0.2	98.85%

Discussion:

The results partially supported our hypotheses across all 6 dimensions (45 total variants).

- conv1_filters:** Accuracy varied only slightly (98.11%–98.45%) with no clear trend. This suggests conv1 filter count is not a bottleneck for MNIST — even 4 filters capture sufficient low-level features.
- conv2_filters:** A clearer trend: accuracy increased from 98.21% (10 filters) to 98.70% (80 filters). Since conv2 feeds directly into the fully connected layers, its capacity has more impact on classification.
- dropout_rate:** Non-monotonic results — 0.2 performed best (98.67%) while 0.5 performed worst (98.01%). Moderate regularization helps, but the relationship is not linear. Our hypothesis that higher dropout would hurt was partially wrong.
- fc1_size:** No strong trend — accuracy peaked at 256 nodes (98.76%) but improvements were modest. This supports our hypothesis that MNIST's bottleneck is in the convolutional layers, not FC.
- batch_size:** Clear monotonic decrease as batch size increased (98.83% at 48 → 96.78% at 256), fully supporting our hypothesis. Smaller batches produce more frequent gradient updates, which helps convergence in only 3 epochs.
- learning_rate:** Best result at lr=0.1 (99.02%), with a sharp drop at lr=0.001 (95.28%) due to insufficient convergence. The optimal range appears to be 0.05–0.1, higher than the baseline of 0.01, suggesting the baseline was somewhat conservative.

Reflection

- Junrui:** This project helped me better understand how CNNs learn features from images. Seeing the filters and how they change the images made the concept much easier to understand.
- I also learned how transfer learning works, and how a pretrained model can be reused for a new task with only a small amount of data.
- One thing that surprised me was how fast the transformer model trained on CPU. Even though its accuracy was a bit lower than the CNN, it still worked quite well on this dataset.
- Junyao:** This project helped me understand more clearly how CNNs and transformers process image data differently. The extension experiments were especially useful because they made the effects of design choices more concrete, such as changing batch size, using a CLS token, or replacing learned filters with Gabor filters. I also found it interesting that a hand-designed first layer could perform almost as well as a

learned one on MNIST. Overall, this project gave me a better practical understanding of how deep networks are built, analyzed, and compared.

Acknowledgement

This project was completed collectively by Junrui Ding and Junyao Han. We referenced the PyTorch documentation and tutorials for network architecture and training loop structure. The Greek letter dataset and **GreekTransform** code were provided by the course instructor. We also used AI tools (Claude Code) for debugging help, implementation guidance, code explanation, and report drafting support.